

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה 10 עמודים

מבנה הבחינה:

בבחינה 3 שאלות.

משך זמן הבחינה 3 שעות.

עליכם לענות על כל השאלות.

משקל השאלות מפורט בגוף השאלון.

שימו לב:

שאלה 3 היא שאלה מפוצלת ל - 2 גרסאות :

אחת מיועדת לסטודנטים שלמדו את הקורס בסמסטר 24ב,

והשניה מיועדת לכל שאר הסטודנטים.

לכן יש להקפיד לענות על השאלה בגרסה הנכונה לפי הסמסטר

בו למדתם את הקורס.

סטודנט שיענה על גרסה לא נכונה של שאלה זו לא יזכה לנקודות!!!

את תשובותיכם רשמו אך ורק במחברת הבחינה

כתבו בכתב יד ברור ומסודר.

חומר עזר המותר בשימוש:

ספר הקורס "Essentials of Programming Languages".

מדריך הלמידה "שפות תכנות".

חומר עזר:

ספר הקורס: essentials of programming languages

מדריך הלמידה.

מותר לכתוב הערות בכתב יד בלבד, רק על גבי מדריך הלמידה.

אסור שימוש בעזרים ובחומרים מקוונים.

אינכם חייבים

להחזיר את השאלון לאוניברסיטה הפתוחה

בהצלחה !!!



שאלה 1 (35 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81.

בשאלה זו נרצה להוסיף לשפה תמיכה בביצוע לולאת foreach המאפשרת לבצע פעולה על סדרת איברים.

להלן הדקדוק המגדיר את לולאת foreach :

$Expression ::= \text{foreach } (Type\ identifier\ in\ [\{ Expression \}^*])\ do$

$Expression$

$\text{foreach-exp } (typ\ id\ exps\ body)$

$Type ::= int \mid bool \mid func$

הסבר על המרכיבים של הביטוי החדש:

לביטוי foreach-exp המרכיבים הבאים :

- **typ** – הוא אחד מהמילים המוגדרות ע"י Type, ומציין את טיפוס הערכים מתוך כלל הערכים שיש במרכיב exps בהם יש לטפל.
המילה int מציינת שמדובר בלולאה המבוצעת רק על ביטויים שערכם מספר שלם מתוך כלל הביטויים ברשימה exps.
 - המילה bool מציינת שמדובר בלולאה המבוצעת רק על ביטויים שערכם בוליאני מתוך כלל הביטויים ברשימה exps.
 - המילה func מציינת שמדובר בלולאה המבוצעת רק על ביטויים שערכם הוא פרוצדורה מתוך כלל הביטויים ברשימה exps.
 - **id** – שם מזהה חדש המוכר רק במסגרת ביצוע הלולאה, ובכל מחזור לולאה תבוצע לו כריכה (binding) לערך הבא המתאים מ- exps.
 - **exps** – רשימת ביטויים כלשהם (מסוגים שונים) שאת ערכם יש לסקור במסגרת הלולאה. אלה הם הערכים אליהם ייכרך id בכל מחזור לולאה (בכל פעם לערך הבא מהסוג שהוצהר)
 - **body** – ביטוי המתאר את גוף הלולאה שיש לבצע.
- ביטוי foreach מחזיר את רשימה (סוג תשובה חדשה בשפה) המכילה את תוצאות פעולתו על האיברים מתוך exps בהם טיפל.

דוגמה להמחשה:

דוגמה :

```
> (run "
  foreach (int i in [10, 20, zero?(8), proc(x) zero?(x)])
  do (i, 2)
" )
(list-val (list (num-val 8) (num-val 18)))
>
```

הסבר הדוגמה:

בדוגמה מוצגת לולאת foreach הפועלת על רשימת ביטויים הכוללת 2 ביטויים שערכם מספר, ביטוי אחד בוליאני, וביטוי אחרון שהוא פרוצדורה. הלולאה מבצעת את גוף הלולאה (במקרה זה הפחתת ערך של 2) רק עבור הביטויים שערכם הוא מספר (כי זה הטיפוס אליו היא הוגדרה). ולכן הערך המוחזר הוא רשימת התוצאות של פעולת הלולאה על האיברים המתאימים שנמצאו ברשימה.

ממשו והטמיעו את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת "וישגור" (שפת PROC).

בתשובתכם, הקפידו להסביר היכן בדיוק נדרשים שינויים ותוספות בחלקי המפרש, ומהם השינויים והתוספות.

לא תתקבל תשובה של תיאור מילולי. יש לממש בקוד את השינויים הדרושים.

הנחיות כלליות לפתרון:

- 1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- 2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- 3) הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
- 4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- 5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- 6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב-terminals, non-terminals ופעולות של סגור (}) או פתוח ({)

שאלה 2 (40 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטויים חדשים המאפשרים עבודה עם אירועים (Events)

אירוע (Event), מאפשר לרשום אליו פרוצדורות אשר יופעלו כאשר האירוע מתרחש (Invoke)

בשאלה זו, נדרש להוסיף יכולת ליצירת אירוע חדש, רישום פרוצדורות לאירוע, והפעלת אירוע. לשם כך נוסיף לשפה 2 ביטויים חדשים, אחד המאפשר יצירת אירוע חדש, השני מאפשר הוספת פרוצדורות לאירוע קיים.

כמו כן, נשנה את מימוש הביטוי הקיים בשפה להפעלת פרוצדורה, כך שיתאים הן להפעלת פרוצדורה והן להפעלת אירוע. התחביר של ביטוי הפעלה לא משתנה אלא רק התנהגותו.

להלן הדקדוקים המגדירים את הביטויים שנדרש לממש:

$Expression ::= event.create()$

event-create-exp()

$Expression ::= \langle Identifier \rangle += \{ \{ Expression \}^+ (:) \}$

event-add-exp(evnt , procs)

הסבר על פעולת הביטויים החדשים:

- ביטוי event-create-exp אמור ליצור ולהחזיר אירוע (event) חדש. (שימו לב, שיש כאן טיפוס נתונים חדש)
- ביטוי event-add-exp מאפשר להוסיף לאירוע evnt שכבר נוצר, אוסף של לפחות פרוצדורה אחת.
- ביטוי call-exp הקיים בשפה ונועד להפעלת פרוצדורה, הוא נשאר עם אותו התחביר, אך מימושו צריך להתאים הן להפעלת פרוצדורה, והן להפעלת אירוע. בהפעלת אירוע מופעלות כל הפרוצדורות שנוספו (נרשמו) לאירוע, הפרוצדורות מופעלות לפי הסדר בו נרשמו. לכל אחת מהפרוצדורות המופעלות נשלח הפרמטר שצויין בביטוי ההפעלה.

נדרש לבדוק ולהדפיס הודעות שגיאה מתאימות כתוצאה משימוש שגוי באירועים.

להלן מספר דוגמאות (תקינות וגם לא תקינות) להמחשת הנדרש:


```

> (run "
    let a=1000
    in
      let dec_x= proc (x) set a= -(a,x)
      in
        let OnClick = event.create()
        in
          begin
            <OnClick> += {dec_x;dec_x};
            (dec_x 100);
            <OnClick> += {dec_x;dec_x};
            (OnClick 50);
            a
          end")
(num-val 700)
>

```

בדוגמה 1 מוגדר משתנה a שערכו 1000, ופרוצדורה בשם dec_x . לאחר מכן מוגדר אירוע בשם $OnClick$, ובהמשך מתווספת (נרשמת) אליו הפרוצדורה dec_x 4 פעמים, בפעם הראשונה היא מתווספת לאירוע פעמיים, ובפעם נוספת עוד פעמיים – סה"כ הפרוצדורה dec_x רשומה 4 פעמים תחת האירוע. (זה בא להמחיש שברישום של פרוצדורות לאירוע, הפרוצדורות מתווספות לרישומים הקיימים כבר לאירוע)

לאחר מכן, מופעלת הפרוצדורה dec_x עם פרמטר 100, דבר שגורם למשתנה a להשתנות ל-900. הפעלה זו באה להדגים שביטוי הפעלה עדיין תומך בהפעלת פרוצדורה. לאחר מכן, מופעל האירוע $OnClick$ עם פרמטר 50, דבר שגורם להפעלת כל הפרוצדורות שנרשמו לאירוע לפעול עם פרמטר 50, התוצאה של כך במקרה זה, היא הקטנת המשתנה a ב-200 (כל פעם הוא קטן ב-50 בעקבות הפעלת 4 הפרוצדורות שנרשמו לאירוע). לכן ערכו של a הוא סה"כ 700. לבסוף מוחזר ערכו של a .

בדוגמה זו ניתן לראות הדגמה של אופן הגדרת אירוע, אופן רישום פרוצדורות לאירוע, ואופן הפעלת אירוע או פרוצדורה.

דוגמה 2:

```
> (run "
  let a=10
  in
    let b=200
    in
      let OnClick = event.create()
      in
        let dec_x = proc (x) set a = -(a,x)
        in
          let add_x = proc(x) set b = -(b , -(0,x))
          in
            begin
              <OnClick> += (dec_x/ proc (y) set a = -(b,y); add_x );
              (OnClick 20);
              -(a,b)
            end")
  (num-val -40)
>
```

בדוגמה 2 מוגדרים 2 משתנים a ו-b, 2 פרוצדורות dec_x ו-add_x, וכן אירוע OnClick באירוע נרשמות הפרוצדורה dec_x, פרוצדורה אנונימית, והפרוצדורה add_x. לאחר מכן, מופעל האירוע OnClick עם פרמטר 20, בהפעלת האירוע מופעלת תחילה הפרוצדורה dec_x שמשנה את ערכו של a ל-10, לאחר מכן, מופעלת הפרוצדורה האנונימית שמשנה את ערכו של a ל-180. לאחר מכן מופעלת הפרוצדורה add_x המשנה את ערכו של המשתנה b ל-220. לבסוף התוכנית מחזירה את ההפרש בין a ל-b ולכן תוצאתה -40.

דוגמה 3:

```
> (run "
  let a=10
  in
    let OnClick = event.create()
    in
      <OnClick> += {}
  ")
```

⊗ ⓧ interp.scm:112:27: event-add-exp: an empty list is not allowed
>

בדוגמה 3 מודגמת הגדרת אירוע, ורישום אליו עם רשימת פרוצדורות ריקה (לא תקין תחבירית) על כן מודפסת הודעת שגיאה מתאימה.

דוגמה 4: ✓

```
> (run "
    let a=10
    in
      let p= proc (x) -(x,5)
      in
        let OnClick = event.create()
        in
          <a> += {p}")
```

⊗ ⊗ *interp.scm:118:23: event-add-exp: a is not an event*
> |

בדוגמה 4 מודגם ניסיון לרשום פרוצדורות ל-identifier שאינו מייצג אירוע, על כן מודפסת הודעת שגיאה מתאימה.

דוגמה 5: ✓

```
> (run "
    let a=10
    in
      let p= proc (x) -(x,5)
      in
        let OnClick = event.create()
        in
          <OnClick> += {p;7}")
```

⊗ ⊗ *interp.scm:156:12: event-add-exp: event cannot contain non-procedure elements*
> |

בדוגמה 5 מודגם ניסיון לרשום לאירוע ביטוי שאינו מייצג פרוצדורה, על כן מודפסת הודעת שגיאה מתאימה.

דוגמה 6:

```
> (run "
  let a=10
  in
    let p= proc (x) -(x,5)
    in
      let OnClick = event.create()
      in
        (OnClick 20)"
```

⊗ ⊗ *interp.scm:97:35: event-call: cannot invoke an empty event*
> |

בדוגמה 6 מודגם ניסיון להפעיל אירוע ריק (אירוע שהוגדר אך לא נרשמו אליו פרוצדורות), על כן מודפסת הודעת שגיאה מתאימה

דוגמה 7:

```
> (run "
  let a=1000
  in
    let dec_x= proc (x) set a= -(a,x)
    in
      let OnClick = event.create()
      in
        begin
          <OnClick> += {dec_x;dec_x};
          (a 100)
        end"
```

⊗ ⊗ *interp.scm:99:20: call-exp: only procedure or event can be invoked*
>

בדוגמה 7 מודגם ניסיון להפעיל ביטוי שאינו מייצג פרוצדורה או אירוע, על כן מודפסת הודעת שגיאה מתאימה


```

> (run "
  let a=1000
  in
  let dec_x= proc (x) set a= -(a,x)
  in
  let OnClick = event.create()
  in
  let OnOpen = event.create()
  in
  begin
    <OnClick> += {dec_x;dec_x;dec_x;dec_x};
    <OnOpen> += {proc (x) if zero?(x) then set a=350 else set a= 550};
    (OnClick 50);
    set OnClick = OnOpen;
    (OnClick 0);
    a
  end")

(num-val 350)
>

```

בדוגמה 8 מודגמת עבודה עם 2 אירועים שונים. מוגדרים האירועים OnClick ו-OnOpen, בתחילה מבוצע רישום פרוצדורות לאירועים אלה, לאחר מכן, מופעל האירוע OnClick עם פרמטר 50, דבר הגורם למשתנה a להשתנות לערך של 800. לאחר מכן, מבוצעת פעולת השמה המשנה את האירוע OnClick להיות האירוע OnOpen, ושוב מופעל האירוע OnClick שהפעם משנה את ערכו של a ל-350. ולבסוף מוחזר ערכו של a. לכן תוצאת הרצת התוכנית היא 350.

ממשו והסמינעו את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת "וירמוז" (שפת Implicit-Refs).

בתשובתכם, הקפידו להסביר היכן בדיוק נדרשים שינויים ותוספות בקבצי המפרש, ומהם השינויים והתוספות.

הנחיות כלליות לפתרון:

- 1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- 2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- 3) הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
- 4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- 5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- 6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור (}) או פתוח ({ }

שאלה 3 (25 נקודות)

שימו לב – זוהי שאלה מפוצלת ל-2 גרסאות שונות. עליכם לפתור את הגרסה המתאימה לסמסטר הלימוד בו למדתם את הקורס. פתרון של גרסה שגויה לא יזכה בניקוד

גרסה עבור סטודנטים שלמדו בסמסטר 24ב

להלן תוכנית בשפת PROC (שפת "וישגור")

```
let y=30
in
zero?((proc(x) -(x,5) -(y,25)))
```

רשמו את שלבי חישוב התוכנית ע"י המפרש תחת סביבה התחלתית ריקה, באמצעות שימוש בסימונים מוסכמים כמתואר למשל בספר הלימוד בעמודים 77-78.

לבסוף רשמו מהי תוצאת התוכנית?

גרסה עבור יתר הנבחנים

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let y=30
in
zero?((proc(x:?) -(x,5) -(y,25)))
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש באלגוריתם שנלמד בפרק 7 בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו האם התוכנית מוטפסת היטב! ואם כן, מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

בהצלחה!